



TRABALHO PRÁTICO 1: OPERAÇÃO CODEBREAKER

Redes de Computadores Abril/2026

Dados do Trabalho

Titulo do trabalho: Trabalho Prático 1: Operação Codebreaker

Alunos: Thales Gregory Vasconcelos Santos – 2018019630

1 Introdução

Este trabalho tem como objetivo apresentar o desenvolver um sistema de quebra de senhas (Cobreaker) baseado na mecânica de tentativa e erro com feedback posicional, utilizando conceitos de redes de computadores, com foco no modelo cliente-servidor via sockets TCP. A implementação foi realizada com base em boas práticas de engenharia de software, buscando garantir clareza, organização e corretude no funcionamento do sistema.

Ao longo do projeto, fora utilizada a implementação de protocolos de comunicação, além de conhecimentos teóricos e práticos relacionados à modelagem do problema, definição de arquitetura, implementação e validação da solução. Além disso, aspectos como tratamento de erros, padronização de comunicação e robustez do sistema foram considerados durante o desenvolvimento.

2 Arquitetura do Sistema

2.1 Visão Geral

O sistema desenvolvido segue o modelo arquitetural cliente-servidor, no qual duas entidades distintas se comunicam por meio de uma rede para a troca de informações. Nesse modelo, o servidor é responsável por centralizar a lógica da aplicação e processar as requisições recebidas, enquanto o cliente atua como interface de interação com o usuário, enviando solicitações e exibindo os resultados retornados.

Como foi solicitado pelo enunciado, utilizamos o protocolo TCP para estabelecer a comunicação entre os módulos. O TCP (Transmission Control Protocol) é um protocolo de transporte de mensagens utilizado para garantir que os dados sejam entregues de forma fidedigna e ordenada. No nosso sistema, a utilização desse protocolo se mostrou imprescindível para garantir a integridade das mensagens trocadas, levando em consideração que manter a completude do conteúdo das mensagens é essencial para o correto funcionamento do sistema de quebra de senhas.

O fluxo da comunicação entre o cliente e o servidor é estruturado da seguinte forma:

- O cliente inicia a comunicação enviando uma mensagem de requisição ao servidor, contendo a tentativa de senha a ser verificada.
- O servidor recebe a mensagem, processa a tentativa de senha e gera um feedback posicional, indicando quais caracteres estão corretos e em suas posições corretas.
- O servidor envia o feedback de volta ao cliente, que o exibe para o usuário, permitindo que ele faça novas tentativas com base nas informações recebidas.

2.2 Estrutura dos componentes

2.2.1 Servidor

O servidor ficou responsável por receber as mensagens de requisição do cliente, processar as tentativas de senha e enviar os feedbacks correspondentes. Ele é implementado utilizando sockets TCP para estabelecer a comunicação com o cliente.

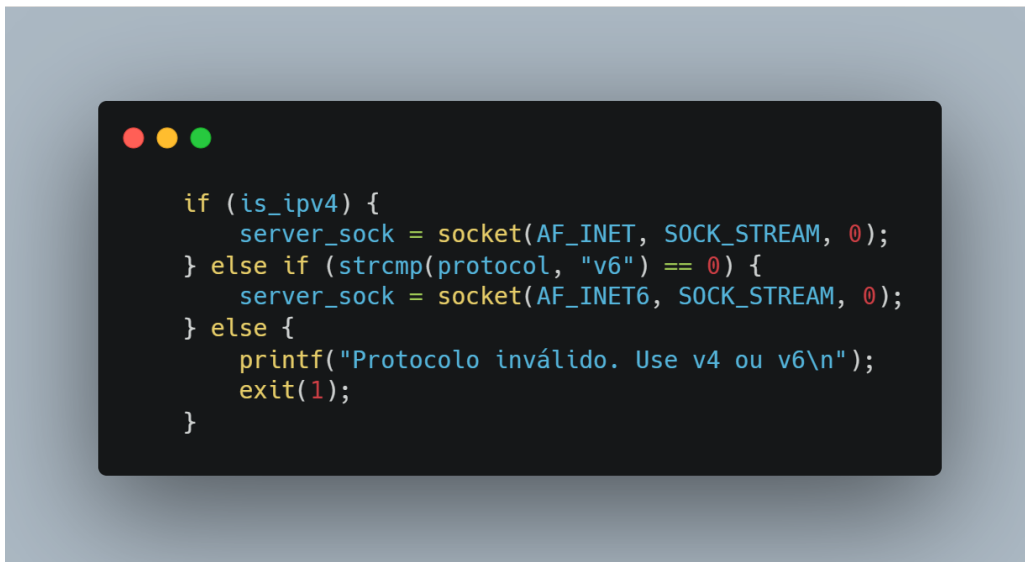


Figura 1: Definição de protocolo de comunicação utilizando TCP pelo servidor.

Na imagem anterior, pode ser observado a utilização das constantes `AF_INET` e `SOCK_STREAM` para configurar o socket TCP. Essas constantes são interpretadas a partir do método `sock()`, da biblioteca `#include <sys/socket.h>`, garantindo a correta configuração do socket para a comunicação TCP.

O servidor foi projetado utilizando o conceito de `passive open`, em que a sequência para aceitar a conexão é realizada por meio dos métodos `bind()`, `listen()` e `accept()`.

O método `bind()` é utilizado para associar o socket a um endereço IP e porta específicos, permitindo que o servidor escute por conexões nessa porta. O método `listen()` é chamado para colocar o servidor em modo de escuta, aguardando por conexões de clientes. Por fim, o método `accept()` é utilizado para aceitar uma conexão de um cliente, retornando um novo socket específico para essa conexão.



Figura 2: Definição do bind no servidor.



Figura 3: Definição do listen no servidor.



Figura 4: Definição do accept no servidor.

Por fim, após a definição do protocolo de comunicação e a configuração do servidor para aceitar conexões, o servidor entra em um loop de processamento, onde aguarda por mensagens de requisição do cliente, processa as tentativas de senha e envia os feedbacks correspondentes. Esse processo se utiliza do método `validate_pass()`, que é responsável por comparar a tentativa de senha recebida com a senha correta e gerar o feedback posicional adequado.

2.2.2 Cliente

Já para o cliente, a implementação se torna bem mais simplificada, utilizando apenas o método `connect()` para estabelecer a conexão com o servidor, e os métodos `send()` e `recv()` para enviar as tentativas de senha e receber os feedbacks do servidor, respectivamente, se aproveitando da struct `HackerMessage` para padronizar o formato das mensagens trocadas entre o cliente e o servidor e garantir a integridade dos dados transmitidos.

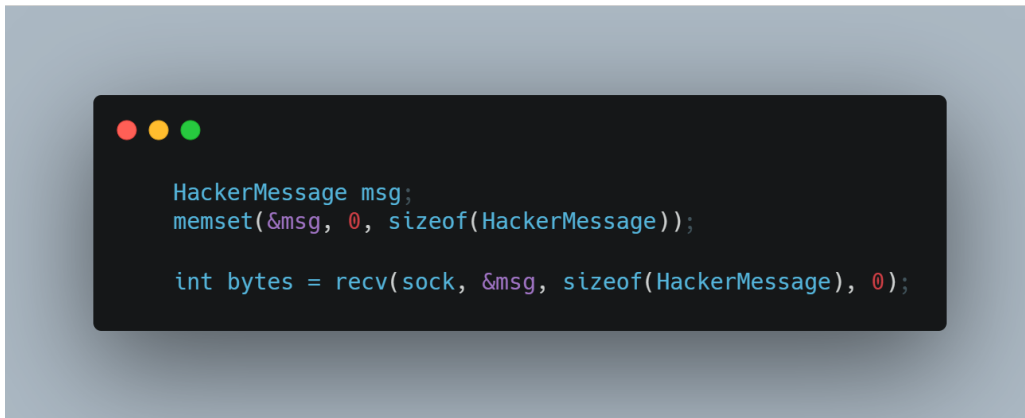


Figura 5: Definição da variável de armazenamento da mensagem de requisição do cliente.

3 Desafios e Dificuldades

Durante o desenvolvimento do sistema, foram enfrentados alguns desafios e dificuldades, principalmente relacionados à implementação da comunicação via sockets TCP e ao tratamento de erros. Num primeiro momento, o entendimento do modelo cliente-servidor e a configuração correta dos sockets se mostrou um desafio, exigindo uma compreensão aprofundada dos conceitos de redes de computadores e do funcionamento dos protocolos de comunicação.

A pouca prática prévia relacionada à programação em C e à utilização de sockets também contribuiu para a dificuldade inicial, demandando um esforço adicional para aprender e aplicar esses conceitos de forma eficaz.

No entanto, após superados os desafios, a fase de padronização das mensagens utilizando o protocolo previamente definido, também gerou uma certa dificuldade inicial, que foi superada com uma maior rapidez do que a fase de implementação da comunicação via sockets TCP, devido à experiência adquirida durante o desenvolvimento do projeto. Entretanto, mesmo com uma certa dificuldade inicial, foi interessante entender a importância de padronizar as mensagens, fazendo com que o restante do desenvolvimento se mostrasse mais tranquilo e fluído.

Por fim, garantimos a robustez do sistema por meio de um tratamento de erros adequado, utilizando mensagens de erros claras e informativas para o usuário, além de implementar mecanismos de validação para garantir a integridade dos dados transmitidos e recebidos.

4 Conclusão

Assim, concluímos que esse projeto permitiu a aplicação prática de conceitos de redes de computadores, programação em C e desenvolvimento de sistemas utilizando o modelo cliente-servidor. A implementação do sistema Codebreaker proporcionou um estudo claro e conciso sobre a comunicação via sockets TCP.

Referências

- [1] DCC/ICEx/UFMG. Especificação do trabalho prático 1 — operação codebreaker. Documento interno, 2026. Material da disciplina.
- [2] GeeksforGeeks. Socket programming in c/c++. <https://www.geeksforgeeks.org/c/socket-programming-cc/>, 2025. Acesso em: 22 abr. 2026.
- [3] Marcos Augusto Menezes Vieira. Redes de computadores. Slides da disciplina DCC218 - INTRODUÇÃO A SISTEMAS COMPUTACIONAIS, Departamento de Ciencia da Computacao, ICEx/UFMG, 2026. Material de aula.